

Prize Bond Checking App — Complete Viva Guide

Final Year Project (CS619) — Flutter + Firebase Study guide written in simple English. Everything here matches your actual source code.

1. Project Overview

Project Title

Prize Bond Checking App — a mobile application for checking Pakistan National Savings prize bond results.

One-line pitch (memorize this)

"My app lets prize bond holders in Pakistan instantly check if their bonds won, get automatic push notifications when they win, save their bond portfolio, see the official draw schedule, and even buy/sell bonds in a marketplace — and the core checking works fully offline."

Problem Statement

- In Pakistan, millions of people own prize bonds issued by **National Savings**.
- To check results, people must find the official PDF lists, download them, and **manually search** their 6-digit bond number in thousands of winning numbers.
- This is slow, error-prone, and many winners **never claim their prize** because they never check.
- There is no easy way to get **alerted automatically** when your bond wins.
- Buying/selling bonds happens informally with no simple platform.

Objectives

1. Instant digital checking of a bond number against draw results.
2. Save a personal bond portfolio ("My Bonds") that is checked automatically.
3. Push notification to every user when a new draw is published, and a **personal congratulation notification** to winners.
4. Work **offline** using a local cache, because internet is not always available.
5. Show the official **draw schedule** with the next draw date for each denomination.
6. Provide a simple **marketplace** to list bonds for sale and contact sellers on WhatsApp.
7. Give an **admin panel** to publish draw results, upload official PDFs, and manage users.

Who can use it

- **Normal users** — bond holders who check results, save bonds, and use the marketplace.
- **Guests** — can browse results and the schedule without an account (My Bonds and Marketplace ask them to sign in).
- **Admin** — publishes draw results, uploads official PDFs, approves/suspends users, views statistics.

Benefits / Real-world application

- Saves time: seconds instead of manual PDF searching.
- No missed prizes: automatic winner matching + push notifications.

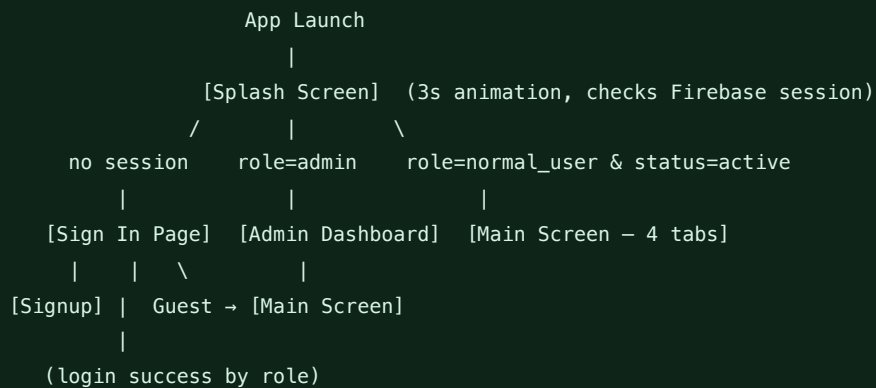
- Works in low-connectivity areas (offline Hive cache).
- One place for results, schedule, portfolio, and trading.

Technologies (one-line each)

Technology	Role
Flutter (Dart)	Cross-platform UI framework — one codebase for Android & iOS
GetX	State management + dependency injection + navigation
Firebase Auth	Email/password login with roles (admin / normal user)
Cloud Firestore	Main cloud database (users, draws, saved bonds, marketplace, notifications)
Firebase Storage	Stores official draw result PDFs uploaded by admin
Firebase Cloud Messaging (FCM)	Push notifications
Cloud Functions (Node.js)	Server-side logic: broadcast new draws, match winners, send pushes
Hive	Local NoSQL database for offline draw cache
GetStorage	Simple key-value local storage (bonds, settings, sync queue)

2. Complete Application Flow

The big picture



MAIN SCREEN (bottom navigation, IndexedStack):

- Tab 0: HOME → quick check, stats, latest draws, Scanner FAB
- Tab 1: MY BONDS → saved portfolio (sign-in required)
- Tab 2: MARKET → marketplace listings (sign-in required)
- Tab 3: DRAWS → schedule + all results + filters + PDF export

From Home header (gear icon) → [Settings] → [Profile] → Sign Out

From Home FAB (camera icon) → [Scanner] → returns number → Quick Check

ADMIN SIDE:

- [Admin Dashboard] → stats, draw list
 - [Create Draw] (publish results + optional PDF upload)
 - [Manage Users] (approve / suspend accounts)

Screen-by-screen (why each screen exists)

1. Splash Screen (`SplashScreen`) - Shows the logo with a 2-second fade + scale animation. - **Why it exists:** gives the app time to check the Firebase session and decide where to send the user. It reads `FirebaseAuth.instance.currentUser`; if logged in, it fetches the user document from Firestore collection `customers` and checks two things: **role** (`admin` → Admin Dashboard, otherwise → Main Screen) and **status** (must be `active`; `pending` / `suspended` users are signed out).

2. Sign In Page (`SignInPage`) - Email + password fields, a **role selector** (Normal User / Admin), a "Create Account" link, and a **Continue as Guest** button. - **Why it exists:** authentication gate. Guest mode was added so people can check results without forced registration (better user experience). - On login the app also checks account **status**: `pending` shows an "awaiting approval" dialog, `suspended` shows a blocked message.

3. Signup Page (`SignupPage`) - Collects first/last name, phone, email, 4-digit PIN, address, city, password. - Creates a Firebase Auth account, then writes a profile document to Firestore `customers/{uid}` with `status: 'pending'`. - **Why pending?** The admin must approve new accounts — a moderation/security feature. - After signup → Confirmation page ("account is pending for approval") → back to Sign In.

4. Main Screen (`MainScreen`) - The shell of the user app: a `BottomNavigationBar` with 4 tabs and an `IndexedStack` body. - **Why IndexedStack?** It keeps every tab alive in memory, so switching tabs does not reload data or lose scroll position/state.

5. Home Screen (`HomeScreen`) - Two stat cards (**Total Bonds** saved, **Next Draw** date), a **Quick Check** card (denomination dropdown + 6-digit number field + Check Result button), the result card (🏆 WINNER / Not a Winner), and the **latest 5 draw results**. - A floating

camera button opens the **Scanner**; the scanned number is filled into the quick-check field automatically. - **Why it exists:** the single most common task (check one bond) should take one screen and one tap.

6. Scanner Screen (`ScannerScreen`) - Live camera view with a dark overlay, animated corner brackets and a moving scan line. - Uses the `mobile_scanner` package to detect QR/barcodes. The detected value must match the regular expression `^\d{6}$` (exactly 6 digits — the Pakistani bond number format); anything else is ignored. - After a valid scan, the camera stops and a result view shows the number with "Use This Number" (returns it to Home via `Get.back(result: number)`) or "Scan Again". - **Why it exists:** typing numbers is error-prone; scanning is faster and avoids typos.

7. Draws Screen (`DrawsScreen`) — Tab 3 - Two sections: an expandable **Draw Schedule** (official National Savings dates per denomination with a computed "next draw" date and countdown, plus **Download Full Schedule PDF**), and the **Draw Results** list. - Results can be **filtered** by denomination chips, city chips, and a from/to date range (bottom-sheet filter). Active filters show as chips; pull-to-refresh reloads. - An **Export PDF** button generates a results report locally from whatever is currently filtered. - When offline, an offline banner shows and results come from the **Hive cache**.

8. Draw Detail Screen (`DrawDetailScreen`) - Shows draw info (number, denomination, date, city, total winners) and all **winning numbers as green pill chips**. - The PDF button has three smart states: **Download PDF** (admin uploaded an official PDF to Firebase Storage), **Generate PDF** (no official PDF → app builds one locally with the `pdf` package), **Open PDF** (already on device).

9. My Bonds Screen (`MyBondsScreen`) - **Auth wall:** guests see a "Sign In / Create Account" screen instead. - Logged-in users see their saved bonds as cards (denomination badge, number, date added, 🏆 WINNER badge, delete icon) and an **Add New** bottom sheet (denomination + 6-digit number). - **Offline-first design:** the bond is saved to **local storage (GetStorage)** immediately, then synced to Firestore (`saved_bonds` collection) in the background when online. Adding bonds never fails because of bad internet.

10. Marketplace Screen (`MarketplaceScreen`) - **Auth wall** for guests. Logged-in users see active listings (denomination, asking price, bond number, seller name and city) with a denomination filter. - **Contact on WhatsApp** opens a `wa.me` chat with the seller using the `url_launcher` package (falls back to a phone call). - **Sell flow:** choose "From My Saved Bonds" (picks one of your bonds, pre-fills the form) or "Add New Bond" → Sell Bond Screen (price, city, WhatsApp number) → **List for Sale**. - **Offline-first:** a new listing is saved to a **local pending queue** first, and automatically pushed to Firestore the next time the app is online.

11. Settings Screen (`SettingsScreen`) - Toggles for **Notifications** and **Auto-Check** (persisted in GetStorage), link to **My Profile**, and **Logout / Sign In**.

12. Profile Screen (`ProfileScreen`) - Shows avatar with initials, full name, email, city, account type (Admin/Normal), status, and member-since date, plus Sign Out with a confirmation dialog. - Reads from `UserController` which keeps a **cached copy of the profile in GetStorage**, so the profile shows instantly even before Firestore responds.

13. Admin Dashboard (`AdminDashboardScreen`) - Three stat cards: **Total Users**, **Saved Bonds**, **Total Draws** (Firestore `count()` aggregation queries — fast and cheap because they don't download documents). - List of all published draws with a delete option, **Upload New Draw Result** button, **View Users** button, Logout.

14. Create Draw Screen (`CreateDrawScreen`) - Admin enters: denomination (dropdown), draw number, draw date (date picker), city, and **winning numbers as one comma-separated text field**. Optionally attaches an **official PDF** using `file_picker`. - On **Publish:** PDF uploads to Firebase Storage (progress bar 0–50%), the draw document is written to Firestore `draws`, then the app **auto-checks all users' saved bonds** of that denomination and marks winners, with staged progress labels ("Uploading PDF → Saving to DB → Auto-checking user bonds"). - Writing to `draws` also **triggers the Cloud Function** that sends push notifications (explained in section 4).

15. Manage Users Screen (`CustomerListScreen`) - Admin sees every user with a status chip and can change status via a bottom sheet: **active / pending / suspended**. This powers the account-approval flow.

3. Feature-by-Feature Explanation

3.1 Prize Bond Checking (core feature)

- **Purpose:** tell the user in seconds whether a bond number won.

- **How it works (step by step):** 1. User selects a denomination (Rs. 100 / 200 / 750 / 1500 / 7500 / 15000 / 25000 / 40000) and enters a 6-digit number. 2. `HomeController.checkBond()` delegates to `DrawController.checkBond(number, denomination)`. 3. The controller loops through the loaded draws and checks: `draw.denomination == denomination && draw.winningNumbers.contains(number)`. 4. If found → `isWinner = true`, a **local notification** fires, and a winner dialog appears showing which draw was won. Otherwise a "Not a Winner" card is shown.
- **Where do the draws come from?** Online → Firestore (`draws` collection, newest first) and simultaneously cached into Hive. Offline → straight from the Hive cache. So **checking works with zero internet**.
- **Expected result:** instant verdict with the winning draw's number and city.

3.2 QR / Barcode Scanner

- **Package:** `mobile_scanner` (uses the device camera + Google ML Kit barcode engine internally under Android; AVFoundation on iOS).
- **Flow:** Camera opens → each frame is analyzed → `onBarcodeDetected` receives a `BarcodeCapture` → the raw value is validated with regex `^\d{6}$` → valid number stops the camera and shows a result view → "Use This Number" returns it to the Home screen where it fills the quick-check field.
- **Why QR instead of OCR?** OCR (reading printed digits with ML Kit text recognition) is unreliable on worn banknotes/bonds and needs heavy ML models. QR scanning is deterministic, fast, and works in poor light. This was a deliberate engineering trade-off (it is even written as a comment in `pubspec.yaml`).
- **Error handling:** a dedicated error view appears if camera permission is denied.

3.3 Draw Results + Filters

- Firestore query: `draws` ordered by `drawDate` descending.
- Filters are applied **client-side** in the `filteredDraws` getter: denomination equality, case-insensitive city substring match, and date range. Client-side filtering means filters also work **offline** on cached data.
- Pull-to-refresh calls `loadDraws()` again.

3.4 Draw Schedule

- The official National Savings schedule is built into the app: Rs.100 monthly (1st), Rs.200 & 750 monthly (15th), Rs.1500 twice a month (1st & 15th), Rs.7500 quarterly (Feb/May/Aug/Nov), Rs.15000 & 25000 quarterly (Jan/Apr/Jul/Oct), Rs.40000 quarterly (Mar/Jun/Sep/Dec).
- The app **computes the next draw date** for each denomination from today's date and shows a countdown badge. Home's "Next Draw" stat uses the same logic.
- **Download Full Schedule** generates a PDF locally (`PdfExportService.exportScheduleToPdf`) and saves it in the app's **Documents directory** so it stays available offline.

3.5 Offline Mode (a key selling point — explain proudly)

- **ConnectivityService** (a GetX service) listens to `connectivity_plus` events and exposes a reactive `isOnline` flag. Wi-Fi, mobile data, or ethernet count as online.
- **Draws:** every successful Firestore load is written into the **Hive box** `draws_cache` (each draw JSON-encoded by id). When offline — or when Firestore fails — the app reads from this cache and shows an "offline" banner.
- **My Bonds:** stored locally first (GetStorage), synced to Firestore in the background.
- **Marketplace:** new listings go into a **pending queue** in GetStorage and are flushed to Firestore automatically on the next online session.
- **PDFs:** downloaded PDFs' file paths are remembered (Hive settings box) so they re-open without re-downloading; the schedule PDF is stored permanently in Documents.
- **Sentence for the viva:** "I used a cache-aside strategy: Firestore is the source of truth, Hive is the offline replica, and writes are queued locally and replayed when connectivity returns."

3.6 PDF Download & Generation

- **Two paths:** 1. **Admin uploaded an official PDF** → the app downloads it from **Firestore Storage** (`refFromURL().writeToFile()`), shows live progress, verifies the file is valid (>100 bytes), and remembers the local path. 2. **No official PDF** → the app **generates one itself** with the `pdf` package: a styled A4 document with the draw header and the winning numbers laid out in a 6-column grid.
- Files are opened with `open_filex` (hands the file to the phone's PDF viewer). Draw-results export and the schedule PDF use the same generation service.

3.7 My Bonds (portfolio) + Auto-Check

- Validation: exactly 6 digits, numeric only, no duplicates (same number + denomination).
- Bond IDs are `uid_timestamp` so they are unique per user.
- Saved to GetStorage instantly → background sync inserts into Firestore `saved_bonds` (with a duplicate check on `userId` + number + denomination) and increments the user's `savedBondsCount`.
- **Auto-check happens in two places:**
- **When the admin publishes a draw**, the admin flow calls `autoCheckBondsForDraw` which queries all `saved_bonds` of that denomination, marks matches with `isWinner: true` + `winningDrawId` (batch update), and the Cloud Function sends the winners a personal push.
- The My Bonds screen can also run a local re-check of saved bonds.

3.8 Marketplace

- Firestore collection `marketplace`; query: `isActive == true`, ordered by `listedAt` descending, limit 50, optional denomination filter.
- Listing document contains: seller UID/name/city/phone, bond number, denomination, asking price, listing date, active flag.
- When posting, the seller's display name is fetched from their `customers` profile.
- **WhatsApp contact:** builds `https://wa.me/<phone>?text=...` (phone normalized to +92 format) and opens it with `url_launcher`; fallback is a `tel:` call.
- **Why WhatsApp instead of in-app chat?** Everyone in Pakistan already uses WhatsApp; building chat would add server cost and complexity without adding trust. (Good "scope decision" answer.)

3.9 Notifications (explain this one carefully — examiners love it)

There are **two cooperating systems**:

A. Push notifications (FCM + Cloud Functions) — server side 1. On login, the app gets the device's **FCM token** and saves it to the user's `customers` document (also refreshed automatically on token rotation). 2. When the admin publishes a draw, Firestore trigger `onNewDraw` (Cloud Function, `onDocumentCreated("draws/{drawId}")`) fires and does two things: - **Broadcast:** queries all `active` normal users with valid tokens and sends "📄 New Draw Results Available" pushes in **chunks of 500** (FCM's multicast limit) via `sendEachForMulticast`. - **Winner matching:** compares the draw's winning numbers against every saved bond of that denomination; each winner gets a **personal 🎉 Congratulations! You Won!" push** plus an in-app notification document (with a duplicate-prevention check so a winner is never notified twice for the same draw+bond). - **Token hygiene:** invalid/expired tokens reported by FCM are removed from Firestore. 3. Tapping a notification navigates by payload type: `draw_result` → Draws screen, `winner` → My Bonds.

B. Local notifications (flutter_local_notifications) — device side - One Android channel: id `prize_bond_channel`, name "Prize Bond Notifications", max importance. - Used to **display** FCM messages when the app is in the foreground or background (the background handler `firebaseMessagingBackgroundHandler` is a top-level function annotated `@pragma('vm:entry-point')` — required so it survives when the app process is killed), and for instant win alerts during a manual check. - Foreground messages also show an in-app snackbar.

App states covered: foreground (local notification + snackbar), background (background handler shows notification), terminated (`getInitialMessage()` handles the tap that launched the app).

3.10 Authentication & Roles

- **Firestore Auth email/password.** Passwords are never stored by the app — Firebase handles hashing and verification.
- Signup creates the Auth user + a Firestore profile in `customers` with `status: 'pending'`.
- Login flow: authenticate → fetch profile → reject if missing → gate on status (`pending` → dialog + sign-out, `suspended` → blocked) → verify the selected role matches the profile role → save FCM token → initialize notifications → cache profile → route (admin vs user).
- Friendly error messages for every `FirebaseAuthException` code (wrong password, user not found, too many requests, etc.).
- **Session policy:** a login timestamp in `GetStorage` with a 1-hour validity window.
- **Multi-user hygiene:** if a different account logs in on the same device, `UserController` wipes the previous user's local bonds and caches first — prevents data leaking between accounts.

3.11 Admin Features

- Publish draw (with PDF upload to `draw_results/{denomination}/{drawNumber}.pdf` in Storage), delete draw (also deletes its Storage PDF), dashboard statistics via **Firestore** `count()` **aggregations**, user approval/suspension.
- A callable Cloud Function `getAdminStats` exists as a secured server-side alternative — it rejects non-authenticated callers and checks `role == 'admin'` in Firestore before returning counts.

4. Firebase — How Each Service Is Used

Service	Used for	Key detail to mention
Firestore Auth	Email/password login, guest browsing, sign-out	UID links the user to all their Firestore data
Cloud Firestore	Collections: <code>customers</code> , <code>draws</code> , <code>saved_bonds</code> , <code>marketplace</code> , <code>notifications</code>	NoSQL document DB, real-time capable, offline-friendly
Firestore Storage	Official draw PDFs uploaded by admin	Path pattern <code>draw_results/{denomination}/{drawNo}.pdf</code>
Cloud Messaging (FCM)	Push notifications (broadcast + personal winner alerts)	Tokens stored per user; invalid tokens auto-cleaned
Cloud Functions	<code>onNewDraw</code> (broadcast + winner match), <code>onUserSignup</code> (hook), <code>getAdminStats</code> (secure callable)	Server-side = works even when users' apps are closed

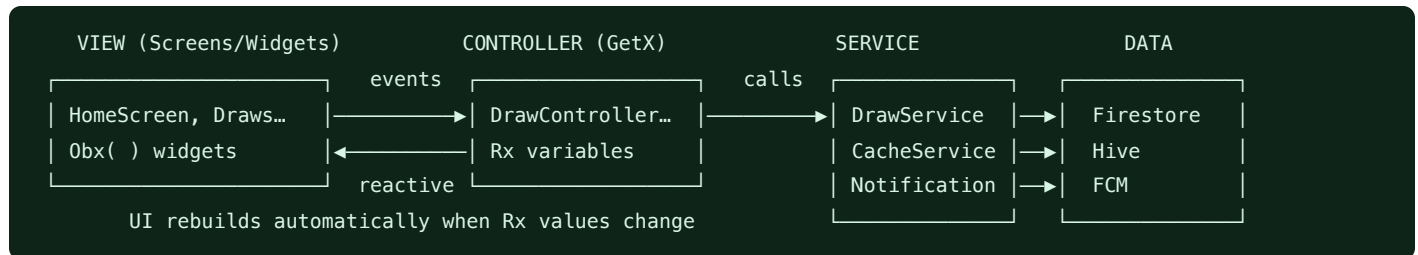
Why Firebase? (memorize)

"Firebase gives me a complete, production-grade backend — authentication, database, file storage, push notifications, and serverless functions — without building or hosting my own server. It has a generous free tier, official Flutter SDKs, real-time listeners, and it scales automatically. For a solo FYP, it let me focus on the app logic instead of server maintenance."

Firestore data model (know your collections): - `customers/{uid}` : firstName, lastName, email, contactNumber, role (`admin` / `normal_user`), status (`pending` / `active` / `suspended`), city, address, pinCode, fcmToken, savedBondsCount, createdAt. - `draws/{id}` : denomination, drawNumber, drawDate, city, winningNumbers (list of strings), pdfUrl/pdfStoragePath/pdfFileSize/pdfUploadedAt, createdAt, uploadedBy. - `saved_bonds/{id}` : userId, bondNumber, denomination, savedAt, isWinner, winningDrawId. - `marketplace/{id}` : sellerUid, sellerName, sellerCity, sellerPhone, bondNumber, denomination, askingPrice, listedAt, isActive. - `notifications/{id}` : userId, title, body, type (`draw_result` / `winner`), relatedId, isRead, createdAt.

5. Architecture

Pattern: MVC with GetX (Model–View–Controller + a Service layer)



- **Models** (`lib/models/`) — plain Dart classes with `fromFirestore` / `toFirestore` / `toJson` mapping (e.g. `DrawModel`, `UserModel`, `MarketListingModel`).
- **Views** (`lib/View/`) — screens and widgets. They contain **no business logic**; they read reactive state via `Obx` / `GetBuilder` and call controller methods.
- **Controllers** (`lib/Controllers/`) — GetX controllers that hold observable state (`.obs` variables) and business logic (e.g. `DrawController`, `AuthController`, `MarketplaceController`).
- **Services** (`lib/Services/`) — a layer under controllers that talks to external systems: `DrawService` (Firestore), `OfflineCacheService` (Hive), `NotificationService` (FCM + local), `ConnectivityService`, `PdfExportService`, `SavedBondService`.

Why GetX? (memorize)

"GetX gives me three things in one lightweight package: **reactive state management** (`.obs` + `Obx` rebuilds only the widget that watches the value — no `setState` over the whole screen), **dependency injection** (`Get.put` registers a controller once, `Get.find` retrieves it anywhere — no `BuildContext` needed), and **navigation** (`Get.to`, `Get.offAll`, snackbars, dialogs and bottom sheets without context). Less boilerplate than Provider or BLoC, ideal for a solo project."

Dependency injection & app startup (`main.dart` order matters)

1. `WidgetsFlutterBinding.ensureInitialized()`
2. `Firebase.initializeApp()`
3. `GetStorage.init()`
4. `OfflineCacheService.init()` (opens the three Hive boxes — must happen **before** controllers, because `DrawController` reads the cache on startup)
5. `NotificationService.initLocalNotifications()` (channel created before any notification can fire)
6. Register the FCM background handler
7. `AppController().initializeController()` — registers permanent controllers with `Get.put(..., permanent: true)`: `ConnectivityService`, `DrawController`, `UserController`, `AuthController`, and others. Screen-specific controllers (`HomeController`, `ScannerController`, `MarketplaceController` ...) are created lazily by their screens with `Get.put` when the screen opens.
8. Lock portrait orientation → `runApp(PrizeBondApp())`.

Navigation

- `GetMaterialApp` with `home: SplashScreen()`; navigation is **imperative GetX**: `Get.to()` (push), `Get.offAll()` (replace whole stack — used after login/logout so the user can't go "back" into the wrong state), `Get.back(result: ...)` (pop with a return value — how the scanner sends the number back).

Responsive UI

- `flutter_screenutil` with design size **400 × 812**: all sizes written as `.w`, `.h`, `.sp` scale proportionally on any phone.

Local storage strategy (two tools, two jobs)

- **Hive** — structured cache of draw documents (fast binary NoSQL boxes: `draws_cache`, `saved_bonds_cache`, `app_settings`). Chosen because it handles many records efficiently.
 - **GetStorage** — tiny key-value store for settings, the saved-bond list, profile cache, and the marketplace pending queue. Chosen because it is synchronous and effortless for small data.
-

6. Technical Deep Dive (simple language)

Folder structure

```
lib/
├─ main.dart           → entry point, initialization order
├─ firebase_options.dart → FlutterFire platform config
├─ Config/             → constants (notification channel, date formats)
├─ Controllers/        → GetX controllers (state + logic)
├─ Services/           → Firestore / Hive / FCM / PDF / connectivity
├─ models/             → data classes with JSON/Firestore mapping
├─ Theme/              → colors + Material 3 theme + text styles
├─ Utils/              → reusable widgets (AppHeader, StatCard, form fields...)
├─ View/               → all screens, grouped by feature
└─ firebase-functions/functions/index.js → Cloud Functions (Node.js)
```

State management in one paragraph

A variable becomes reactive by adding `.obs` (e.g. `RxBool isLoading = false.obs`). Any widget wrapped in `Obx(() => ...)` that reads that variable **rebuilds automatically** when it changes. Controllers own the variables; views only watch them. This is more efficient than `setState` because only the small watching widget rebuilds, not the whole screen.

JSON parsing / model classes

Every model has factory constructors: `DrawModel.fromFirestore(data, docId)` converts a Firestore document (with `Timestamp` fields) into a Dart object; `toFirestore()` converts back. For Hive/offline, `toLocalJson()` / `fromLocalJson()` use ISO date strings because Hive stores plain JSON, not Firestore Timestamps.

Error & exception handling

- Every Firebase call sits in `try/catch`; `FirebaseAuthException` codes map to human-friendly snackbar messages.
- Network failures **degrade gracefully**: draws fall back to the Hive cache with the message "Could not refresh. Showing cached data."
- PDF downloads verify file size and handle `object-not-found` by falling back to local generation.
- Corrupted cache entries are skipped, not crashed on.
- The `logger` package prints structured debug logs during development.

Performance & memory

- `Obx` = minimal rebuilds; `IndexedStack` = no tab reloading; `ListView.builder` -style lazy lists.
- Firestore `count()` aggregation for admin stats (counts on the server — no documents downloaded).
- FCM sends are chunked x500 server-side; notification queries are `limit(50)`.
- Controllers dispose their resources (`onClose()` disposes the camera controller).
- Images/fonts bundled locally; portrait lock avoids rebuild churn.

Security

- Passwords handled entirely by Firebase Auth (never stored in the app).
- Role + status gating at login **and** at the splash screen.
- Admin-only Cloud Function checks `request.auth` + the caller's role in Firestore **on the server**, which the client cannot fake.
- Local data of the previous user is wiped when a different account signs in.
- (Say if asked: "Firestore security rules restrict reads/writes per collection by auth UID and role — and I would tighten them further in production.")

7. Known Limitations & Honest Answers (IMPORTANT — read once)

Examiners respect honesty. If they spot these, answer calmly:

1. **Some duplicate/legacy files exist** (e.g. an older `DashboardPage`, an unused `ScheduleScreen`, two My Bonds controllers).

"Yes — the project went through a refactor from an early prototype to the current MainScreen architecture. The live code path is Splash → MainScreen with four tabs; the old files are kept as reference and would be removed in cleanup. I can show you exactly which files are live."

2. **Schedule data is defined in the app, not fetched from a server.**

"The National Savings schedule is fixed by government policy (same dates every year), so I embedded it and compute the next draw date dynamically. Moving it to Firestore Remote Config is listed in future work."

3. **The My Bonds tab's local auto-check currently uses sample data; the real winner matching runs when the admin publishes a draw** (both client-side batch update and the Cloud Function).

"Winner detection is production-real on the publish path — the Cloud Function and the admin flow both match saved bonds against the new draw. The local re-check button was a demo aid and is planned to point at the cached draws."

4. **Session length is 1 hour by timestamp.** > "A simple session policy; Firebase tokens auto-refresh, so this is an extra app-level control."
5. **No in-app payments in marketplace.** > "Deliberate scope decision — the marketplace connects buyer and seller via WhatsApp; escrow/payments are future work."

8. Future Improvements (say 3–4 confidently)

1. Fetch draw results automatically from the National Savings website (web scraping / official API) instead of admin entry.

2. Real OCR scanning of printed bond numbers with ML Kit text recognition, in addition to QR.
3. In-app chat and secure payments/escrow for the marketplace.
4. Multi-language support (Urdu) and dark mode.
5. Firestore security-rules hardening + App Check.
6. Move the draw schedule to Remote Config; add analytics and Crashlytics.